

Wärtsilä Report

By Kritagya Panthi
+47 940 333 92

June 2026

Project 1: Website and State-of-Charge Algorithm

Introduction

My first project in Wärtsilä was to build a locally hosted web application for data handling, such that a person could simply drag in their data into the application, choose their parameters, and get the plot they wished for. The code is shown in Appendix A. As line 102-167 of the code shows, the application also includes an algorithm made to track the spikes of the given data.

The Algorithm

The algorithm is built by hysteresis filtering the given data to remove all fluctuations in the data that are less than a given threshold. Thereafter, the peaks in the hysteresised data are found. The peaks sequentially are then spikes in the data that make clear the changes that are larger than the threshold.

Hysteresis Filtering

Hysteresis filtering is when small fluctuations in the data are removed [1]. Figure 1 presents an example.

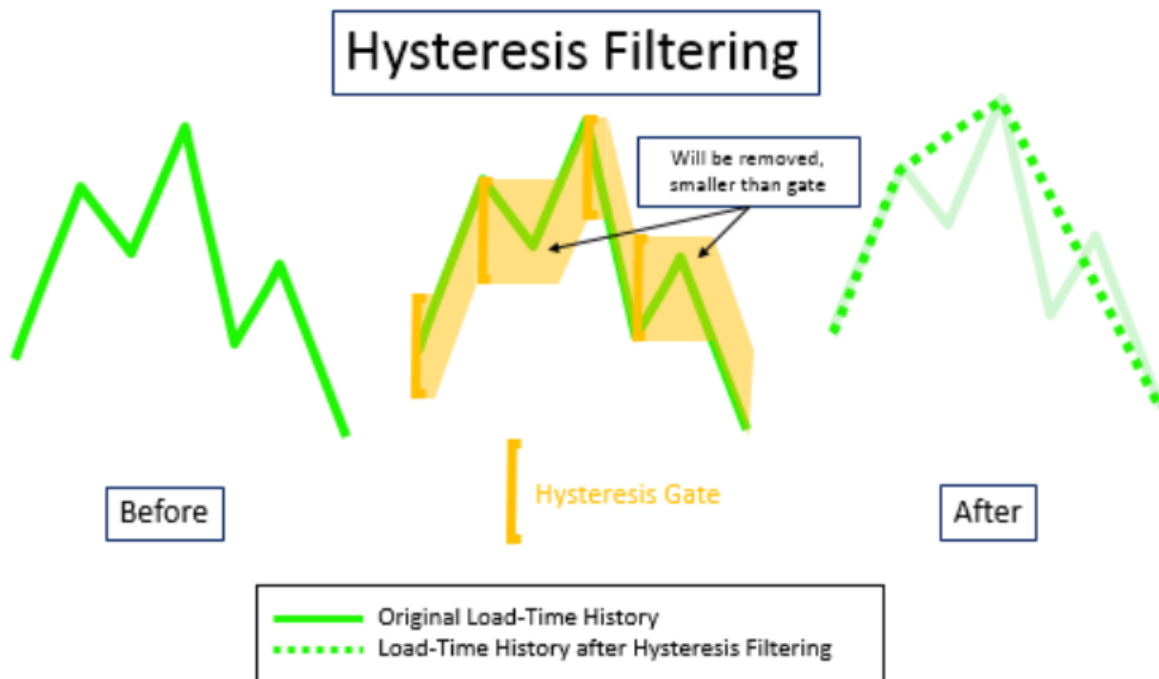


Figure 1: Hysteresis filtering, from [1].

As Figure 1 and Siemens' article explains, the filtering is done by defining a hysteresis gate. Any fluctuations in the data that are less than the hysteresis gate are then removed from the data, as the figure's before and after load-time history demonstrates [1]. The value of the hysteresis gate is what was previously referred to as the threshold.

Peaks in Data

To find the peaks in the data, both for hysteresis filtering and for finding the peaks of the hysteresised data, SciPy's method for finding peaks was utilized [2]. Because SciPy's method for finding peaks does not consider edge data, the algorithm manually considers each edge also as peaks. Figure 2 illustrates where peaks are on a plot of data.

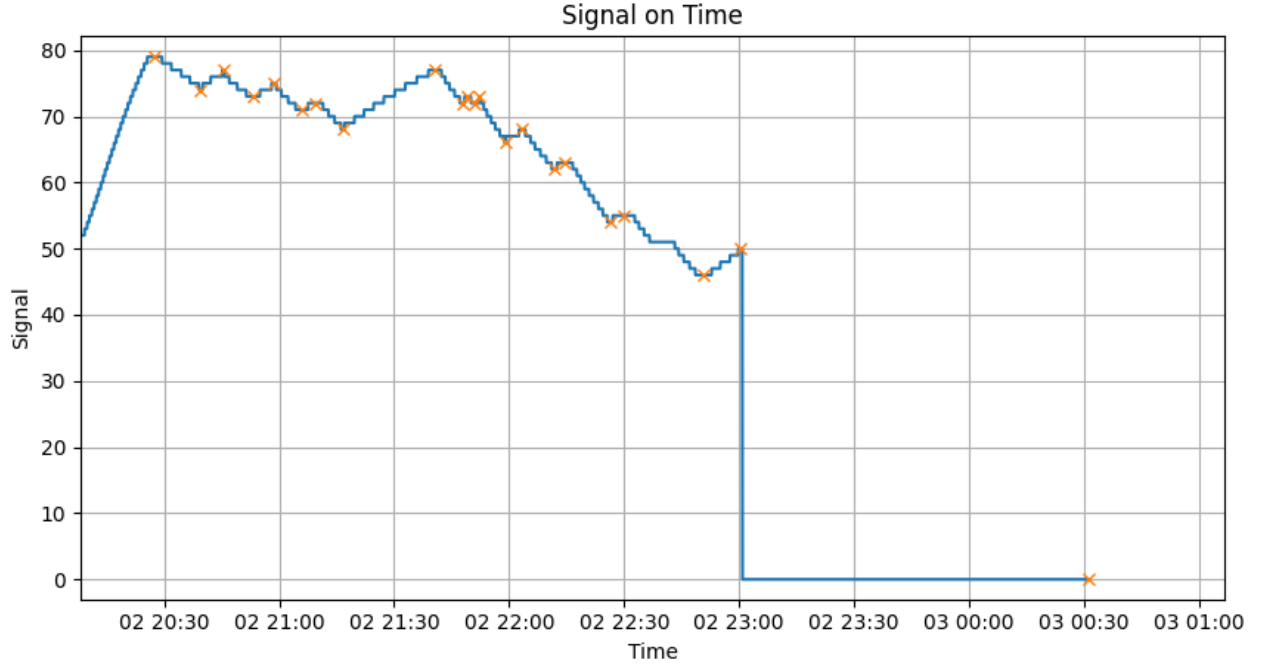


Figure 2: Plot of peaks in a dataset.

The blue graph is the data itself, where the orange x-es represent the peaks. The plot shown is not all of the data and peaks, only the last portion. That is why the end of the graph is considered an edge (and therefore a peak), and not what optically seems like the beginning of the graph.

Visualization of the Algorithm

After finding the peaks of the hysteresised data and sequentially plotting them along with the data itself, one should get a plot that is along the lines of what Figure 3 shows. The chosen threshold was 5.0.

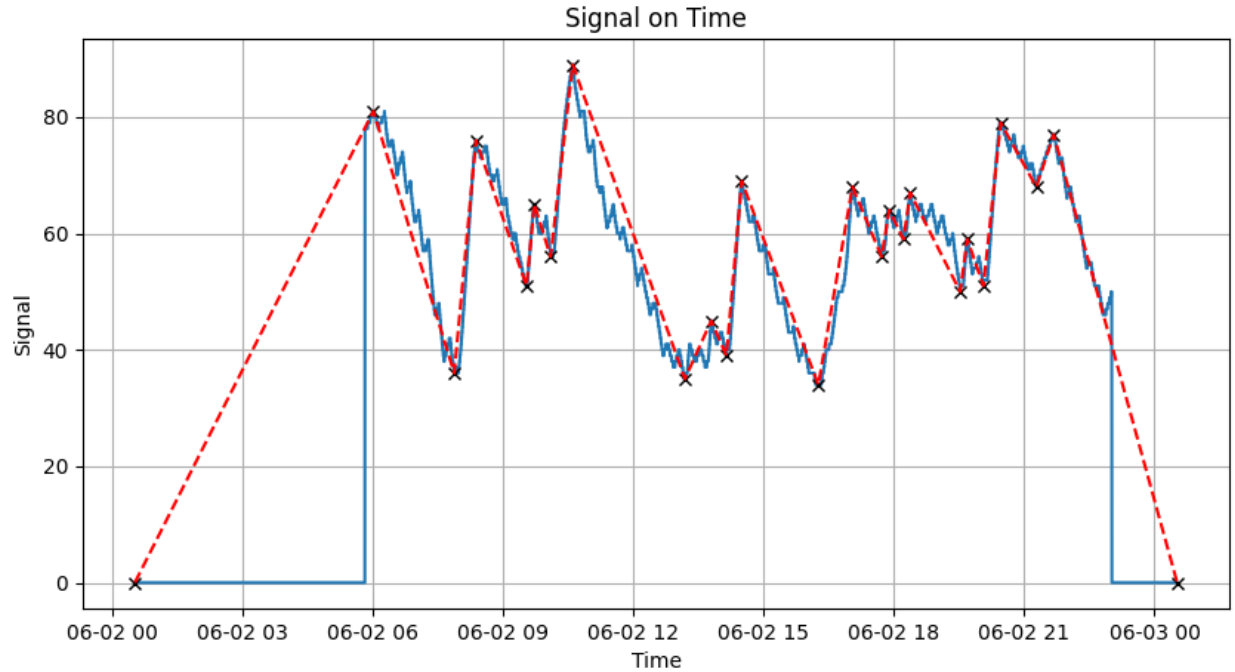


Figure 3: A plot of the data and the hysteresised data's peaks.

The blue graph is the data itself, the black x-es the peaks of the hysteresised data, and the red dashed line a plot of the peaks, sequentially.

As the hysteresised data filtered out all fluctuations which are less than the threshold, the peaks which are found to said data now sequentially must have a distance larger than threshold. Therefore, plotting the peaks leads to a clear illustration of all changes in the data that are larger than threshold.

Guide to the Data Handling Web Application

File Format

The web application expects the data file to be formatted in a specific way. Figure 4 presents an example of how the file could look.

```

Time;Name_A;Name_A;Name_B;Name_C;Name_D;Name_E;Name_F;Name_G;Name_H;Name_I;Name_J;Name_K;Name_L;Name_M;Name_N;Name_O
2016-06-02 00:21:59.871;65533;65533;65532;65534;0;0;65535;0;0;2;0;65535;0;0;0;0
2016-06-02 00:22:00.890;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:01.906;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:02.922;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:03.931;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:04.940;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:05.945;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:06.957;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:07.969;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:08.980;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;1;0;0
2016-06-02 00:22:09.986;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:10.993;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:12.000;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:13.006;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:14.017;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:15.029;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:16.036;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:17.040;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:18.054;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:19.065;65533;65534;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:20.072;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:21.085;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:22.096;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0

```

Figure 4: An example of a data file.

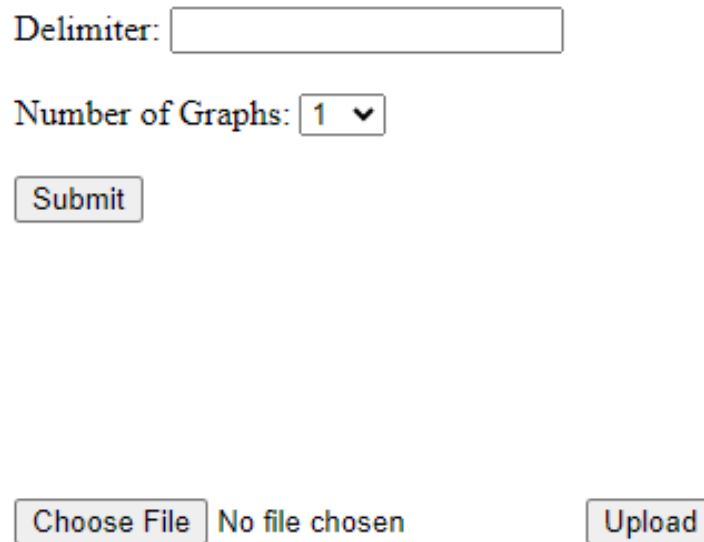
In this case the data file is a text file, which is not an obligation. In addition, the delimiter is a semicolon, but any delimiter suffices.

The headers must be on the top row and have unique names. Additionally, if there is date-time formatted representation of time, it must be as YEAR-MONTH-DAY HOURS:MINUTES:SECONDS.MILLISECONDS. The program will attempts to interpret date-time values which are written differently than stated, but there is no guarantee that it will be interpreted correctly or interpretable at all.

It is important that the data file itself is a plain text file format and not a binary file format. For example, if the user wishes to analyze a .xlsx file, it must be saved as a .csv file before uploaded to this web application.

The First Page

Figure 5 demonstrates how the web application looks when initially opened.



Delimiter:

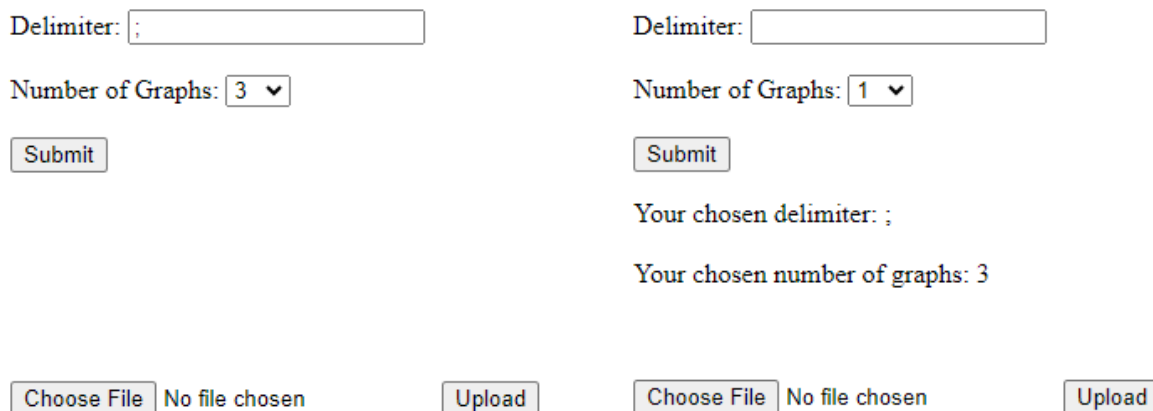
Number of Graphs: ▼

No file chosen

Figure 5: The first page of the web application.

As can be seen, the page contains a text input for the data's delimiter, a dropdown list of the number of graphs one wishes to be presented, a submit button, and finally a field to upload the data file.

Submitting the delimiter and number of graphs is the first step. Figure 6 illustrates how the website should look before (left) and after (right) submitting.



Delimiter: ; Delimiter:

Number of Graphs: 3 ▼ Number of Graphs: 1 ▼

 Your chosen delimiter: ;

 Your chosen number of graphs: 3

No file chosen No file chosen

Figure 6: The website before and after submitting inputs.

As is noticeable, the web page will reflect which delimiter and how many graphs were chosen by the user. If no delimiter and number of graphs is submitted, the program is set to choose ',' as the delimiter and make one graph.

The data file can now be chosen and uploaded, leading to the second page.

The Second Page (/upload)

Figure 7 presents how the second page of the web application might look.

Was unable to turn cell (1 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (2 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (3 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (4 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (5 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (6 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (7 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (8 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (9 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (10 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
Was unable to turn cell (11 down, 78 right) to a number. This value will be ignored in plotting. Cell value:
More cells will be ignored in plotting. These 10 above are examples of which values
File is done being read

Graph nr. 1
Choose your x-axis:
Choose your y-axis:

Graph nr. 2
Choose your x-axis:
Choose your y-axis:

Graph nr. 3
Choose your x-axis:
Choose your y-axis:

Figure 7: The second page of the web application.

The page begins by informing the user which values will further be ignored due to their inability to be represented as a number. The position of these values in the data file, along with the values themselves, are explicitly stated. Figure 8 illustrates how the position of the values are calculated for, in this case, a text file where the delimiter is a semicolon.

```

Time;Name_A;Name_A;Name_B;Name_C;Name_D;Name_E;Name_F;Name_G;Name_H;Name_I;Name_J;Name_K;Name_L;Name_M;Name_N;Name_O
2016-06-02 00:21:59.871;65533;65533;65532;65534;0;0;65535;0;0;2;0;65535;0;0;0;0;0
2016-06-02 00:22:00.890;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:01.906;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:02.922;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:03.931;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:04.940;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:05.945;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:06.957;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:07.969;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:08.980;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;1;0;0
2016-06-02 00:22:09.986;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:10.993;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:12.000;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:13.006;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:14.017;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:15.029;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:16.036;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:17.040;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:18.054;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:19.065;65533;65534;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:20.072;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:21.085;65533;65533;65533;65534;0;0;65535;0;0;0;0;65535;0;0;0;0
2016-06-02 00:22:22.096;65533;65533;65532;65534;0;0;65535;0;0;0;0;65535;0;0;0;0

```

CELL (1 DOWN, 1 RIGHT)
 CELL (4 DOWN, 3 RIGHT)
 CELL (11 DOWN, 6 RIGHT)

Figure 8: How the position of a value is calculated.

It is thereby simple to realize that the data file uploaded on the first page, leading to the screen on Figure 8, had an empty 78th column. This is because the messages shows that every value ignored is 78th in every row, and the value itself is presented as blank.

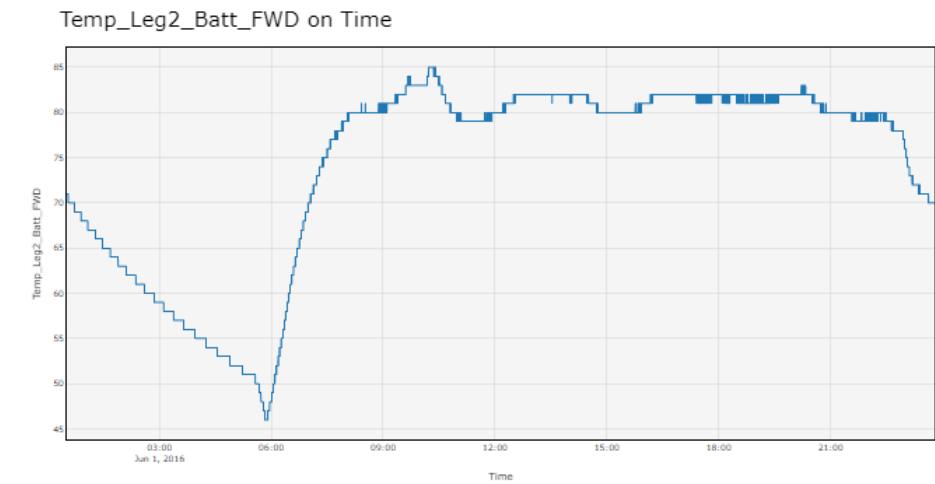
It is notable that these messages will only show to the user if there are values in the data file that the program cannot interpret.

Further, Figure 8 shows that the second page continues. Graph numbers along with dropdown lists of the headers are given to choose an x-axis and y-axis per graph. The number of graphs presented are decided by the number of graphs the user chose on the first page. The user should now choose which columns of data they wish to see the relation of for each graph, and finally submit.

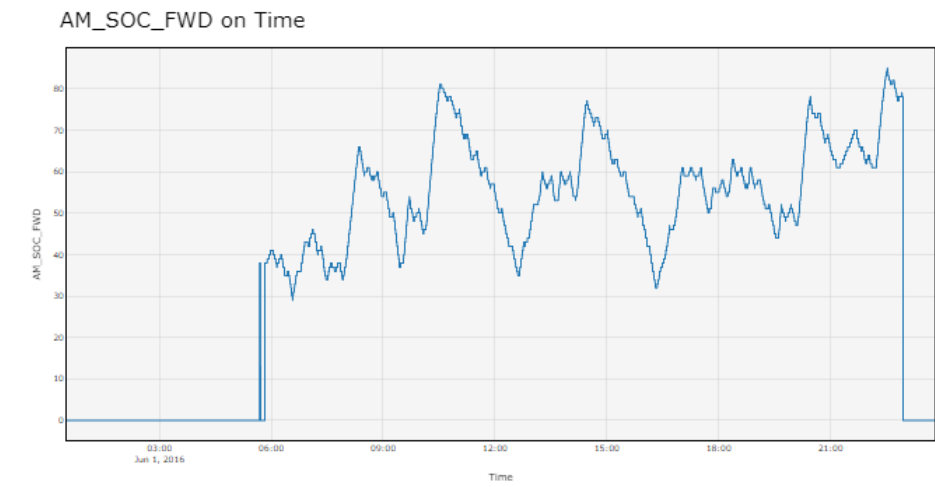
It is of importance that the user does not submit a graph where date-time formatted values are on the y-axis, as the program will not understand how to plot such a graph.

The Third Page, The Fourth Page (/plot, /soc)

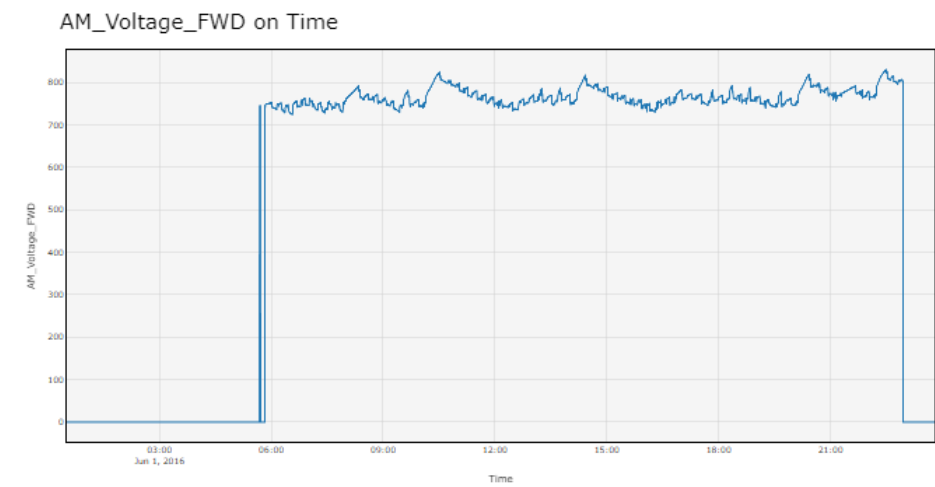
Figure 9 demonstrates how the third page of the web application could look if, on the previous page, the user picks 'Time' for all the x-axes and 'Temp_Leg2_Batt_FWD', 'AM_SOC_FWD', and 'AM_Voltage_FWD' respectively for the y-axes.



Do you wish to analyze the **spikes in the data?**
Threshold:



Do you wish to analyze the **spikes in the data?**
Threshold:



Do you wish to analyze the **spikes in the data?**
Threshold:

Figure 9: The third page of the web application.

In the top right corner of each graph, the user should notice that there are tools that can be used to download the plot as a png, zoom in and out of the plot, and pan around the plot.

Larger numbers in the data file will be much lower numbers on the plots. This is due to the program's ability to handle 16-bit values where it has been ignored that the first bit is a sign bit. Simply, this means any number over 32767 is interpreted to be 65536 lower than that number.

Under each graph, there is an option to analyze the spikes of the plot. To analyze the spikes, the program needs a threshold to decide the minimum change in the y-axis that registers as a spike. Figure 10 provides an example of the output, if the user wished to analyze the spikes of 'AM_SOC_FWD on Time' on Figure 9. The threshold is set to 5.

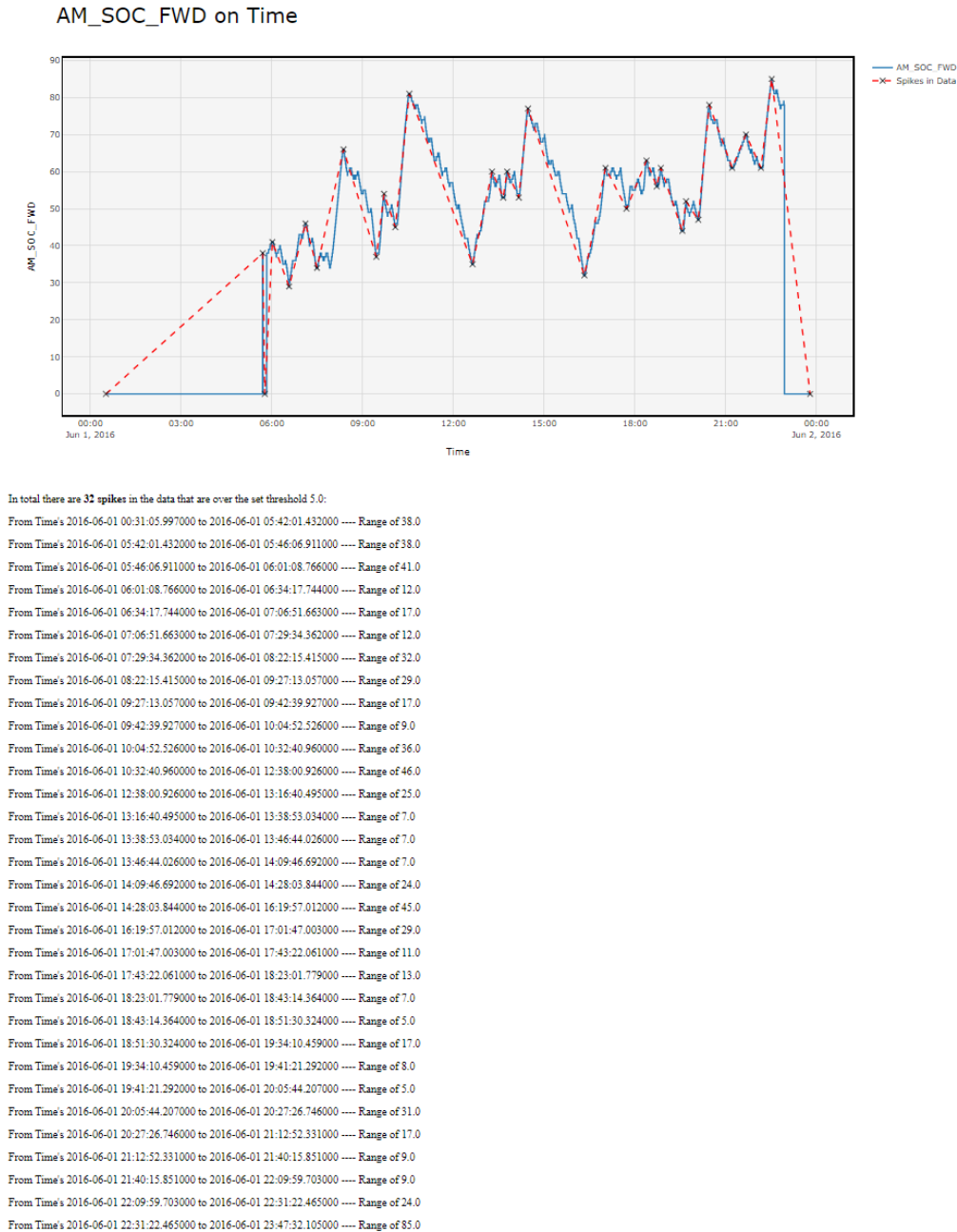


Figure 10: The fourth page of the web application.

As can be seen, a graph is provided with all the spikes in the data that are over the set threshold. Additionally, messages are sent to the user informing how many spikes there are and where they are located on the plot.

Conclusion

The developed web application functions well and is very intuitive in its use. The code was uploaded to a Raspberry PI Model 4 v2 [3] such that the code ran automatically while connected to the Wärtsilä Wi-Fi, so anyone connected to the Wi-Fi can navigate to `http://Rpi4-IP-adress:80` to use it.

Although the implementation fulfills its purpose, improvements could be made in future versions. From a security perspective, user authentication could be introduced to restrict access to authorized personnel. Additionally, stricter validation of uploaded files and protection against malicious file uploads would improve the robustness of the application. From a usability perspective, support for additional file formats such as Excel files, improved error reporting, and more advanced plotting options would enhance the user experience.

Project 2: Lifetime Modeling of Batteries

Introduction

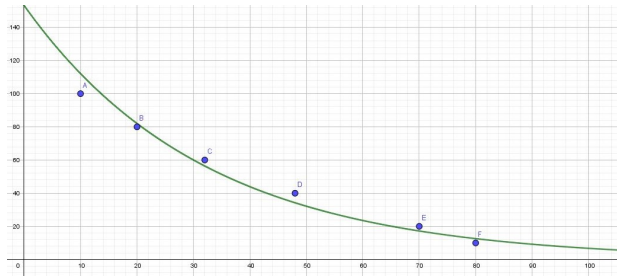
My second summer project for Wärtsilä consisted of developing an algorithm for estimating the State of Health (SOH) of batteries. The project required research into the importance and impact of the different factors that effect SOH, and building my algorithm around that research. The final code is demonstrated in Appendix C.

Simple Exponential Model

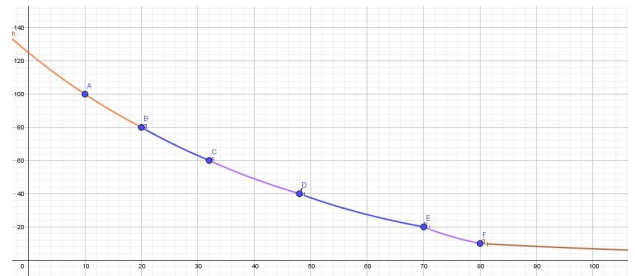
Firstly, there was developed a simple State of Health (SOH) estimation model to gain a preliminary understanding of the problem attempting to be solved. The algorithm was simply based the battery's depth-of-discharge variation (ΔDOD) and state-of-charge variation (ΔSOC). Given the parameters in Table 1, the plots in Figure 11 were made using exponential curve fitting [4]. The y-axis is ΔDOD , and the x-axis cycles until end-of-life (EOL). EOL is presumed to be when the battery falls from 100% to 80% SOH.

Table 1: Parameters for the preliminary SOH estimation model.

Cycles to EOL ($\times 10^3$)	ΔDOD (%)
10	100
20	80
32	60
48	40
70	20
80	10
200	1



(a) EOL estimation using common exponential fitting.



(b) EOL estimation using individual exponential fitting.

Figure 11: Comparison of the two fitting approaches.

As Figure 11 shows, (a) is fitted using a common exponential function between all parameters, while (b) is fitted using different exponential functions between the consecutive parameters. The code presented in Appendix B demonstrates the whole model, along with other assumptions taken, such that there is a linear relationship between how much health is lost and how many cycles there are of that ΔDOD .

The resulting algorithm naturally does not provide an accurate estimate of a battery's SOH, as the model parameters were not derived from any data and several simplifying assumptions were made. However, the purpose of the exercise was not to develop a good SOH estimation model, but rather to gain a deeper understanding of the problem we are wishing to solve.

Research

Researching the different factors and parameters that influence the SOH of lithium-ion batteries, it became clear that battery degradation is affected by a large number of interacting variables, and that different lithium ion batteries can have different variables more relevant to them. To begin, Lithium iron phosphate batteries (LiFePO₄) (LFP) were considered due to their reliable properties often making them a default choice. Other batteries were to be considered later, when the model is ready for scale. For LFP batteries, the factors temperature, charge rate (c-rate), and Δ DOD were studied. As a result, two journal articles were mainly used in developing the model.

Numerical analysis of capacity fading for a LiFePO₄ battery under different current rates and ambient temperatures

Liang et al. investigate LFP battery degradation under different c-rates (1C - 6C) and ambient temperatures (-10°C - 50°C) [5]. Figure 12 demonstrates the resulting figure, with capacity loss (SOH = 100% - capacity loss) after 100 cycles with 100% Δ DOD on the y-axis, and ambient temperature on the x-axis. Each curve corresponds to an integer c-rate between 1C and 6C. From bottom to top, the curves represent 1C, 2C, 3C, 4C, 5C, and 6C, respectively.

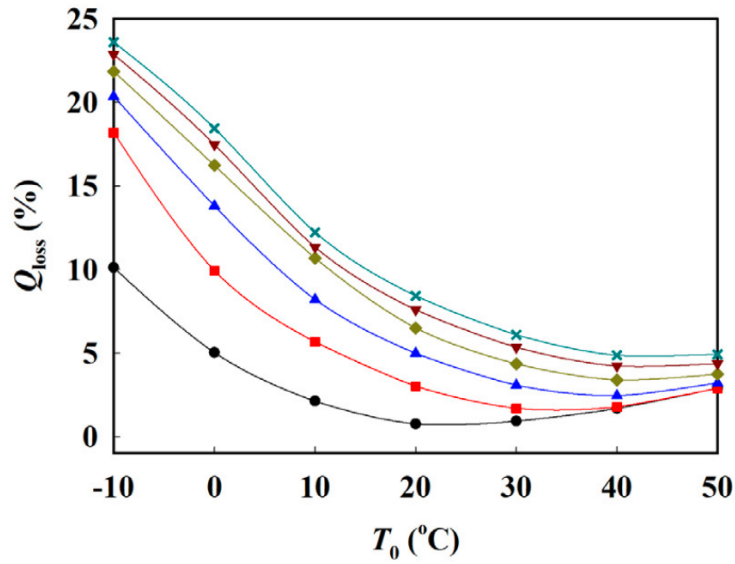


Figure 12: LiFePO₄ battery capacity fading under different c-rates and ambient temperatures, exposed to 100 cycles with 100% Δ DOD.

Liang et al. give sufficient parameters that shows how different temperatures and c-rates affect the health of the battery, but are missing how different Δ DOD levels affect the battery health.

Cycle ageing analysis of a LiFePO₄/graphite cell with dynamic model validations: Towards realistic lifetime predictions

Sarasketa-Zabal et al. study LFP battery degradation under different Δ DOD, where the ambient temperature and c-rate are held constant [6]. Figure 13 shows the affect of different Δ DOD at 30°C and 1C, where capacity retention (SOH = capacity retention) is the y-axis, and full-equivalent-cycles (FEC) is the x-axis.

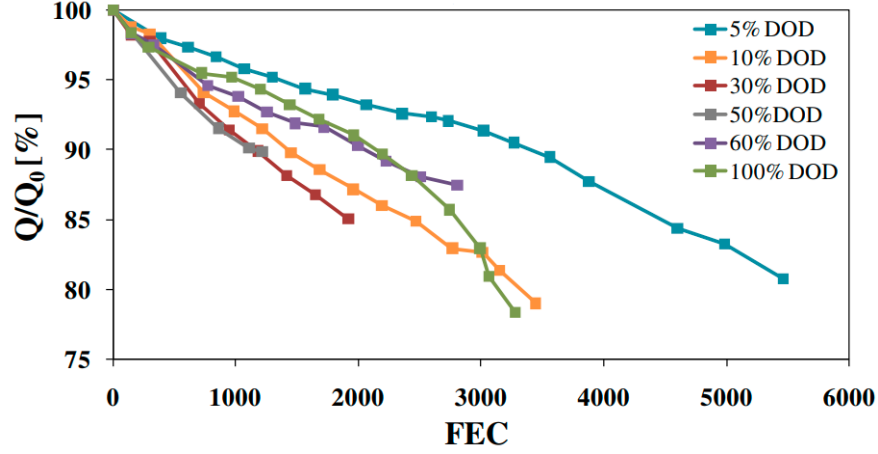


Figure 13: Depth-of-discharge (Δ DOD) influence on LiFePO4 battery degradation, expressed as capacity retention over full-equivalent-cycles (FEC). 30°C, 1C and 50% middle SOC.

Figure 14 shows the effect of Δ DOD when the capacity retention on Figure 13 reaches 90%.

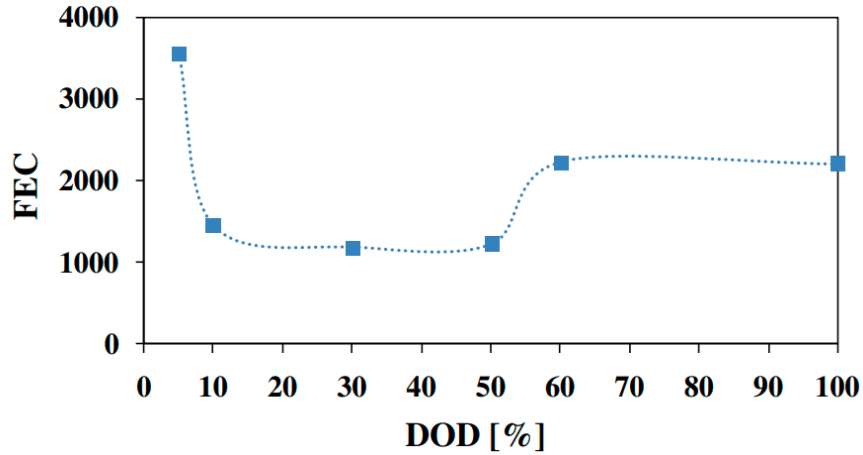


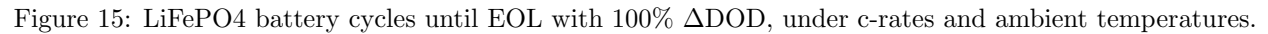
Figure 14: Number of full-equivalent-cycles (FEC) that an LiFePO4 battery could deliver at various Δ DOD levels before reaching ca. 90% SOH. 30°C, 1C and 50% middle SOC.

Sarasketa-Zabal et al. provide good parameters that show how different Δ DOD in charging cycles leads to varying loss of health for the battery.

The Algorithm

To construct the SOH estimation algorithm, the findings and parameters from the papers mentioned above must be manipulated to provide a healthy SOH estimation. In addition, other factors I have not explicitly stated, such as how charging typically provides more damage to a battery than discharging, will be considered

Based on Figure 12, Figure 15 was made with an x-axis of ambient temperature from -10°C to 50°C , and a y-axis of number of cycles until EOL. The different curves are similarly different c-rates.


$$\frac{20\%}{\frac{Q_{loss}(\%)}{100}} \quad (1)$$

to calculate cycles until EOL, as we assume 20% capacity loss is EOL given SOH goes from 100% to 80%. Calculating (2) for all points along a c-rate in Figure 12, we get 7 data points we can graph, which we do by polynomial curve fitting [7]. Not too large polynomial degrees were chosen to avoid overfitting, but to ensure the graph still follows the trend. The data points were in times split in regions, with each region curve fitting independently. Adjacent regions shared a common boundary data point, ensuring that the curves connected.

15

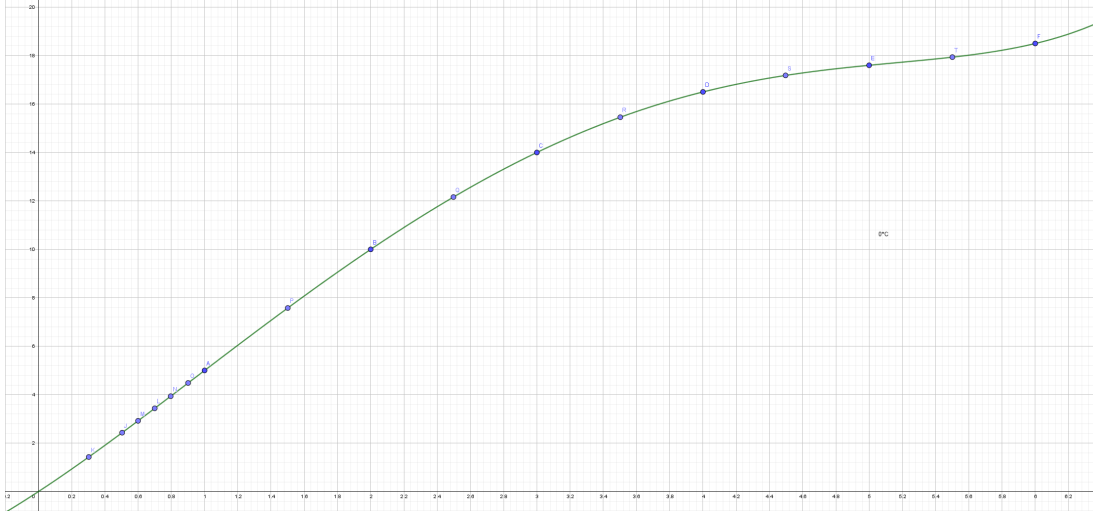


Figure 16: LiFePO4 battery capacity fading under different c-rates at a chosen ambient temperature 0°C , exposed to 100 cycles with 100% ΔDOD .

When done to all data points along all the chosen temperatures in Figure 12, we have the $Q_{loss}(\%)$ values for the other c-rates to plug into (2), which can be plotted in Figure 15. Notice that there are many c-rate values under 1C in Figure 16. Batteries typically charge with less than 1C, meaning we wish for accurate discrete values in these points.

Based on Figure 13 and 14, the graph in Figure 17 was made. Number of cycles until EOL on the y-axis, and varying ΔDOD on the x-axis.

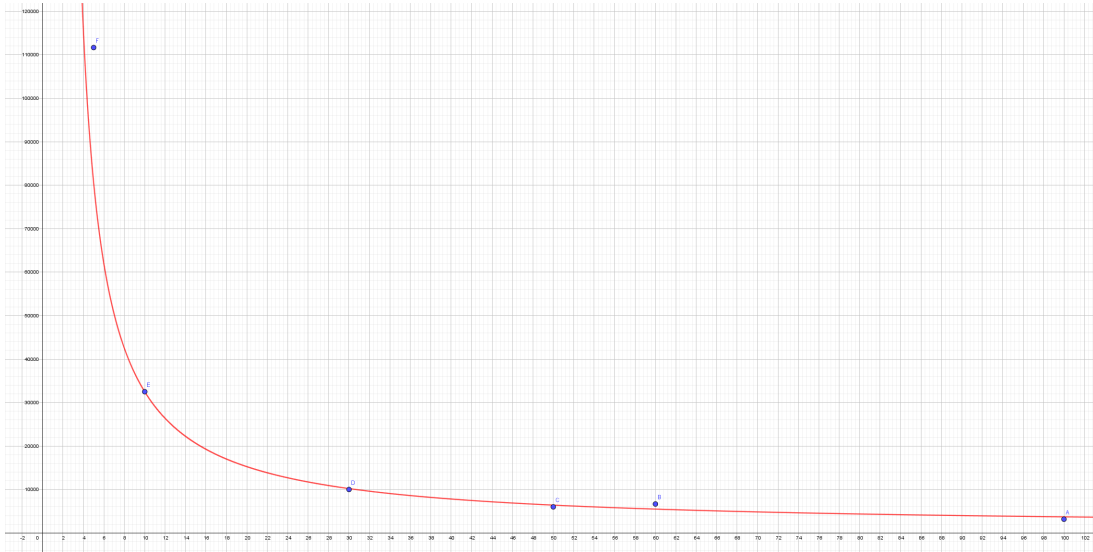


Figure 17: Number of cycles until EOL, as a function of ΔDOD .

As we approximated a linear relationship between number of cycles and SOH loss for 100% ΔDOD , Figure 13 shows that this approximation is also reasonable for the other ΔDOD values. Using Figure 14, number of cycles until EOL was then calculated by translating FEC to pure cycles, then

$$\frac{\frac{20\%}{10\%}}{\# \text{ Cycles until } 90\% \text{ SOH}} \quad (3)$$

$$2 \cdot \# \text{ Cycles until } 90\% \text{ SOH}. \quad (4)$$

Model Implementation

Primarily around Figure 15 and Figure 17, the algorithm was constructed. The main loop to estimate the SOH is seen in lines 259 - 297 of Appendix C, which will be explained. The loop begins as

```

1 #THE MAIN LOOP TO CALCULATE LOSS OF SOH using the previously defined functions
2 for d in range(daysOfCycleProfile):
3     for i in range(len(real_y_peaks)-1):
4         delta_SOC = real_y_peaks[i+1] - real_y_peaks[i]
5         charge = True
6         if delta_SOC < 0:
7             charge = False
8
9         delta_SOC = abs(delta_SOC)
10        delta_t = real_date_peaks[i+1] - real_date_peaks[i]
11        delta_t = delta_t.total_seconds() / 3600
12
13        #Calculated C-rate
14        C_rate = delta_SOC/(100*delta_t)

```

Code 1: Beginning of the main SOH estimation loop.

The two nested for loops define how many times the loop will run. If we imagine the cycle profile shown in Figure 18, `daysOfCycleProfile` is how many days we wish to simulate Figure 18, while `range(len(real_y_peaks)-1)` ensures we go through each red spike of the profile each day.

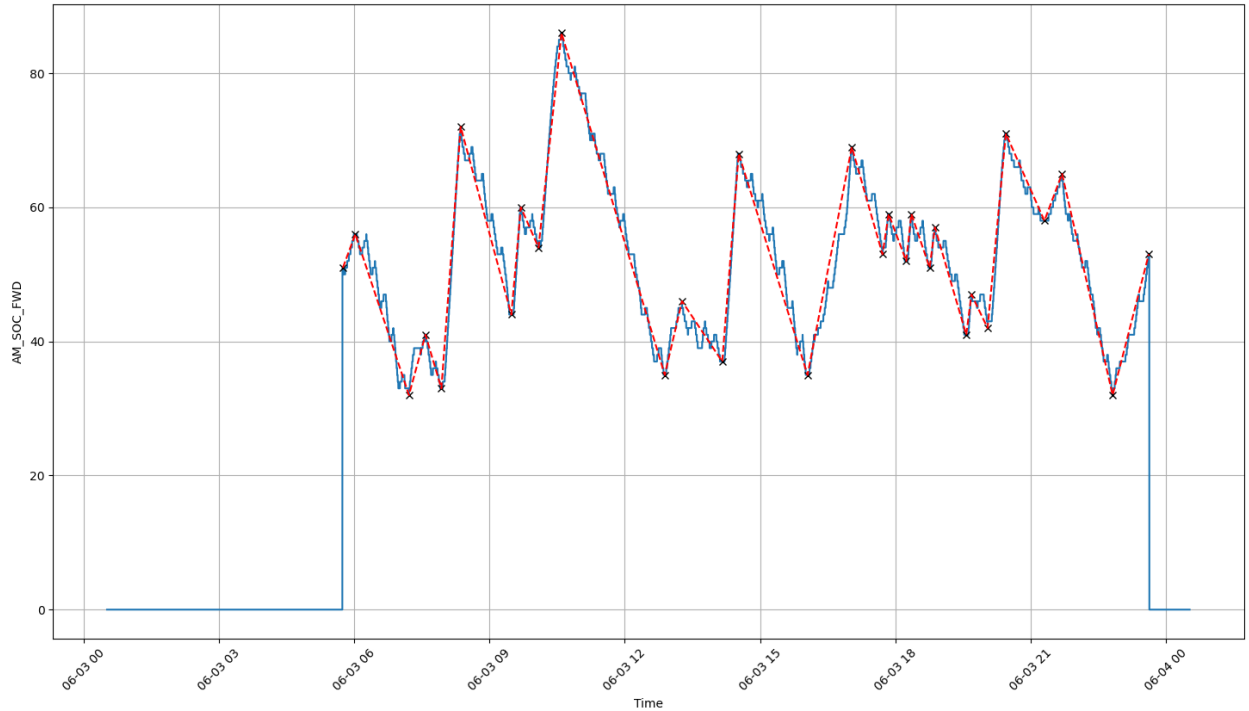


Figure 18: A general battery cycle profile, SOC(%) as function of time

`delta_SOC` is the difference in SOC in the spike, so the y-axis. `delta_t` is the difference in time, so the x-axis. Transforming `delta_t` to hours, the `C_rate` was calculated using the values. The loop continues as

```

1      #Bottom and Upper indexes of the dates during that ΔSOC. This is used to calculate
2      #the mean temperature during that time frame for the cyclesTilEolAt100() function
3      bottom_index_relevant_dates = real_peaks[all_peaks[i]]
4      upper_index_relevant_dates = real_peaks[all_peaks[i+1]]
5
6
7      for j in range(len(Pack_Temp)):
8          relevant_temperatures = Pack_Temp[j][bottom_index_relevant_dates:
9          upper_index_relevant_dates+1]
10         average_temperature = np.mean(relevant_temperatures)
11
12         cyclesTilEolAt100 = CyclesTilEolAt100(C_rate, average_temperature)
13         howManyMoreCycles = CyclesTilEol(delta_SOC) / CyclesTilEol(100)
14         cyclesTilEol = cyclesTilEolAt100 * howManyMoreCycles
15
16         healthLossOfOneCycle = 20 / cyclesTilEol
17         healthLossOfOneChargeOrDischarge = 0
18         if charge:
19             healthLossOfOneChargeOrDischarge = (healthLossOfOneCycle*3)/5
20         else:
21             healthLossOfOneChargeOrDischarge = (healthLossOfOneCycle*2)/5
22
23         healthOfAllPacks[j] -= healthLossOfOneChargeOrDischarge
24 print(healthOfAllPacks)

```

Code 2: Continuation of the main SOH estimation loop

It finds the indexes of the first and last date of the spike in question. Then, for each pack of batteries in the Forward of the ship, the temperatures within that date frame are extracted and found the mean of in `average_temperatures`. For example, if we consider Figure 19 as the varying temperature of pack 1, within the given time frame of the spike, the mean temperature is found.

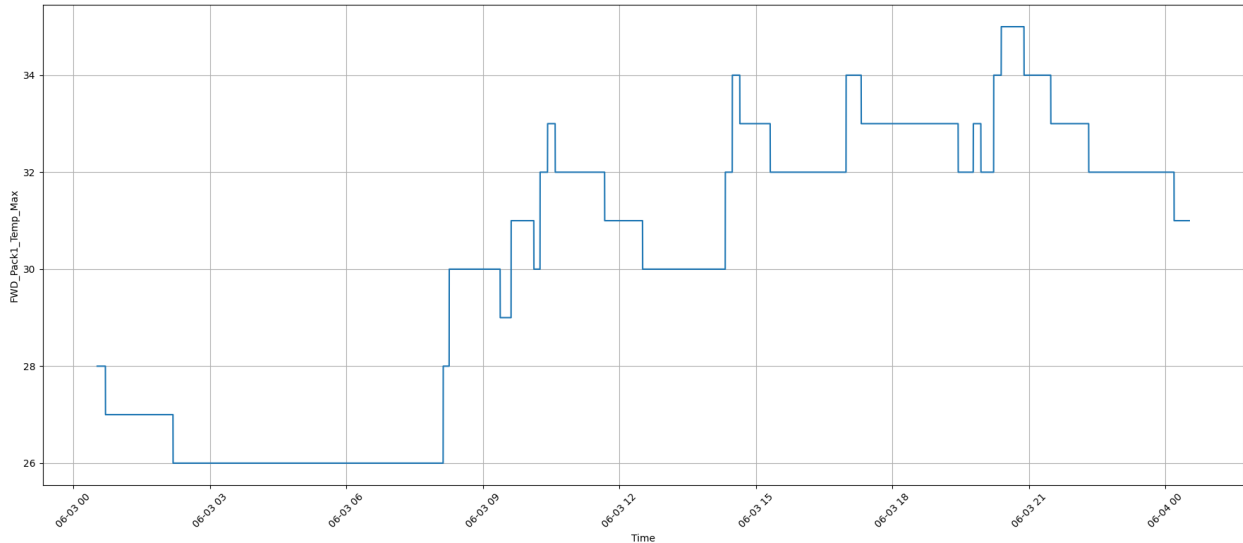


Figure 19: A general temperature (°C) profile of a battery pack

From the average temperature and calculated c-rate, it is plugged into the function `CyclesTilEolAt100()`, which returns cycles until EOL if the ΔDOD is 100%, from Figure 15. Notice that Figure 15 does not have continuous c-rate data, meaning the c-rate given to the function will snap to the discrete value closest to it.

As ΔDOD is not 100%, `howManyMoreCycles` calculates the ratio of `CyclesTilEOL(delta_SOC)/CyclesTilEOL(100)`, where `CyclesTilEOL()` calculates cycles until EOL given a ΔDOD , from Figure 17. This ratio is multiplied by `cyclesTilEolAt100` to calculate `cyclesTilEol`.

`healthLossOfOneCycle` is calculated by continuing the approximation of linearity between number of cycles and SOH loss. `healthLossOfOneChargeOrDischarge` is then finally calculated, where we have

$$\text{healthLossOfOneCycle} = \text{healthLossOfOneCharge} + \text{healthLossOfOneDischarge} \quad (5)$$

where

$$\text{healthLossOfOneCharge} = \frac{3}{2} \text{healthLossOfOneDischarge} \quad (6)$$

which leads to lines 16 - 20 of Code 2. Notice that the check for if the spike is a charge or discharge is in lines 5 - 7 of Code 1. This found value is then subtracted from the SOH of the relevant battery pack.

Finally, when for every simulated day of the cycle profile, each SOC spike and each battery pack is considered in the computation, it prints the remaining health of all the battery packs.

Conclusion

The developed algorithm does not function as well as wished for, and predicts the death of batteries too early. Improvements to the code can be made by:

- More accurate data on lower c-rates should have been used, instead of interpolating the data from c-rates higher than 1C. The c-rates utilized to find relevant values for the model are high, and damage the battery more than for with c-rates lower than 1C. Therefore, I can imagine that there is not as smooth of a transition from lower to higher c-rates as shown in Figure 16, for example. The problem was that this data was difficult to find good documentation on.
- More discrete values should have been interpolated for lower c-rates. As I only interpolated down to 0.3C, many real c-rates are forced to snap up to 0.3C.
- Assumption that health loss is linearly correlated with number of cycles is too optimistic. Although Figure 13 shows its a close approximation, the values in the figure are only applicable for 30°C, 1C, and 50% middle SOC. Realistic data varies much more. The problem was that I could not see another way to approach finding the loss per cycle.
- The graphs in Figure 15 and Figure 17 could have directly been implemented with health loss per cycle on the y-axis, instead of until EOL. This would lead to the calculations being cleaner, instead of forcing the algorithm to jump back to health loss per cycle in line 15 of Code 2.
- There must be a better way than line 12 of Code 2 to find out how many more cycles we must have due to ΔDOD being less than 100%.
- Different levels of SOC is also be an important factor to consider in lifetime modeling of a battery, and should perhaps be considered in this algorithm.

Despite the limitations of the current implementation, the project was highly valuable from a learning perspective. Developing the algorithm required research into battery degradation mechanisms, including the effects of temperature, C-rate, depth of discharge, state of charge, and cycle ageing. It provided practical

experience in translating findings from the literature into a working engineering model, while highlighting the challenges associated with limited data and forced model assumptions.

References

- [1] Hiatt, J. (2019, August 29). *Rainflow counting*. Siemens. [Link here](#).
- [2] SciPy (n.d.). *scipy.signal.find_peaks*. SciPy Documentation. Retrieved August 2024, [link here](#)
- [3] Raspberry Pi (n.d.). *Raspberry Pi 4*. Raspberry Pi Products. Retrieved June 2026, [link here](#).
- [4] Jayson, J. (n.d.). *Exponential Regression*. GeoGebra. Retrieved June 2026, [link here](#).
- [5] Liang, J., Gan, Y., Yao, M., & Li, Y. (2021). *Numerical analysis of capacity fading for a LiFePO₄ battery under different current rates and ambient temperatures*. *International Journal of Heat and Mass Transfer*, 165, 120615. [Link here](#)
- [6] Sarasketa-Zabala, E., Gandiaga, I., Martinez-Laserna, E., Rodriguez-Martinez, L. M., & Villarreal, I. (2015). *Cycle ageing analysis of a LiFePO₄/graphite cell with dynamic model validations: Towards realistic lifetime predictions*. *Journal of Power Sources*, 275, 573–587. [Link here](#)
- [7] Phil (n.d.). *Polynomial fit*. GeoGebra. Retrieved June 2026, [link here](#).

Appendix

A Web Application for Data Handling

```
1 #DATA HANDLING WEB APPLICATION SOURCE CODE. The python environment used to run the code is
2 #not of importance. I have used venv to test and built this project. If any imports
3 #have not been accepted, simply pip install it in the terminal. This code must be uploaded
4 #to a server, where it will run. This is so users of the application have easy access to it
5 from flask import Flask, request, session
6 from markupsafe import escape
7 import numpy as np
8 import plotly.io as pio
9 import plotly.express as px
10 import plotly.graph_objects as go
11 import pandas as pd
12 import csv
13 import os
14 import tempfile
15 import time
16 from datetime import datetime
17 import threading
18 from scipy.signal import find_peaks
19
20
21 #EXTRACT DATA: A function which extracts data from a file, which is seperated by the given
22 #delimiter. Note that the file must
23 #not be a csv file, even though it is interpreted as one. It can be any file as long as it
24 #is delimiter seperated properly
25 #and a plain text file format, as i discuss in the README
26 def extract_data(csvfile, delim):
27     header = []
28     data = []
29     messages = []
30
31     csvfile.seek(0)
32     csvreader = csv.reader(csvfile, delimiter=delim)
33     header = next(csvreader)
34     for datapoint in csvreader:
35         values = []
36
37         for value in datapoint:
38             try:
39                 #Try to interpret the value in datapoint as a number or date-time,
40                 ...
41                 if ":" in str(value):
42                     try:
43                         #Try to explicitly convert the value to date-time (faster),
44                         ...
45                         dt = datetime.strptime(value, "%Y-%m-%d %H:%M:%S.%f")
46                     except:
47                         #... If not possible, try implicitly (slower)
48                         dt = pd.to_datetime(value)
49
50                 values.append(dt)
51
52             else:
53                 value_as_float = float(value)
54                 values.append(value_as_float)
55
56             except:
57                 #... If not possible, this value will be overlooked in plotting due
58                 #to being labled "ignore"
59                 values.append("ignore")
60                 if len(messages)<11:
61                     messages.append(f"Was unable to turn cell ({len(data) + 1} down, {len(
62                     values)}) right) to a number. This value will be ignored in plotting. Cell value: {value}
63                     ")
64
65                 elif len(messages)==11:
66                     messages.append("More cells will be ignored in plotting. These 10 above
67                     are examples of which values")
```

```

55         data.append(values)
56
57     messages.append(f"File is done being read \n")
58
59     return header, data, messages
60
61
62
63 #GENERATE DROPDOWN OPTIONS: A function which generates a dropdown list of the list of values
    in headers
64 def generate_dropdown_options(headers):
65     options = ""
66     for header in headers:
67         options += f"<option value='{str(escape(header))}'>{str(escape(header))}</option>"
68         ""
69     return options
70
71 #GENERATE DROPDOWN: Where GENERATE DROPDOWN OPTIONS generates the dropdown list, this
    function specifically generates
72 #the dropdown list on the second page of the application. This is dependant on the number of
    graphs the user wishes for, as
73 #is clear with the for-loop
74 def generate_dropdowns(num_graphs, headers):
75     options=""
76     for i in range(num_graphs):
77         options += f""Graph nr. {i+1} <br>
78                 <label for= "x-axis{i}"> Choose your x-axis:</label>
79                 <select id="x-axis{i}" name="x-axis{i}"> <!-- First dropdown list
80
81                 {generate_dropdown_options(headers)}
82                 </select>
83
84                 <br>
85                 <label for="y-axis{i}">Choose your y-axis:</label>
86                 <select id="y-axis{i}" name="y-axis{i}"> <!-- Second dropdown list
87
88                 {generate_dropdown_options(headers)}
89                 </select>
90
91                 <br><br>""
92     return options
93
94 #BINARY CONVERSION: A function which ensures that values over 32767 in the data file get
    minimized by 65536, as
95 #any values over 32767 must indicate that it was ignored that a 16-bit value contained a
    sign bit at the beginning
96 def binary_conversion(values):
97     for i in range(len(values)):
98         if values[i] > 32767:
99             values[i] -= 65536
100
101
102 #PLOT SPIKES: A function / algorithm which computes from where to where in the plot there is
    a change in the y-axis which is
103 #larger than the given threshold. The function returns the x- and y-values which, when
    plotted, will reveal these spikes of
104 #data which are larger than the threshold. The range of each spike and where this spike
    exists between is also calculated
105 def plot_spikes(threshold, x_values, y_values, x_axis_name):
106     x_values = np.array(x_values)
107     y_values = np.array(y_values)
108
109     upper_peaks, _ = find_peaks(y_values)
110     lower_peaks, _ = find_peaks(-y_values)
111     all_peaks = np.append(upper_peaks, lower_peaks)

```

```

112 all_peaks = sorted(np.append(np.append([0], all_peaks), [len(y_values)-1]))
113 all_y_values_peaks = y_values[all_peaks]
114
115 soc_peaks=[]
116 peak_index = 0
117 first_round = True
118 while peak_index < len(all_peaks):
119
120     if first_round:
121         next_index = peak_index + 1
122         current_peak_index = peak_index
123         soc_peaks.append(all_peaks[current_peak_index])
124         while next_index < len(all_peaks) and abs(all_y_values_peaks[next_index] -
all_y_values_peaks[current_peak_index]) < threshold:
125             next_index+=1
126             peak_index+=1
127
128         first_round = False
129         peak_index += 1
130         continue
131
132     soc_peaks.append(all_peaks[peak_index])
133
134     if peak_index < len(all_y_values_peaks)-1:
135         if abs(all_y_values_peaks[peak_index]-all_y_values_peaks[peak_index+1])<
threshold:
136             next_index = peak_index+2
137             current_peak_index = peak_index
138             peak_index += 1
139             if all_y_values_peaks[current_peak_index+1] > all_y_values_peaks[
current_peak_index]:
140                 while(next_index < len(all_y_values_peaks) and 0 <= (all_y_values_peaks[
next_index]-all_y_values_peaks[current_peak_index]) < threshold):
141                     peak_index+=1
142                     next_index+=1
143                 else:
144                     while(next_index <= len(all_y_values_peaks)-1 and 0 <= (
all_y_values_peaks[current_peak_index]-all_y_values_peaks[next_index]) < threshold):
145                         peak_index+=1
146                         next_index+=1
147
148
149         peak_index += 1
150
151 soc_peaks = np.array(soc_peaks)
152 soc_y_values_peaks = y_values[soc_peaks]
153
154 upper_peaks, _ = find_peaks(soc_y_values_peaks)
155 lower_peaks, _ = find_peaks(-soc_y_values_peaks)
156 all_peaks = np.append(upper_peaks, lower_peaks)
157 all_peaks = sorted(np.append(np.append([0], all_peaks), [len(soc_peaks)-1]))
158
159 messages = f"In total there are <b>{len(all_peaks) - 1}</b> spikes</b> in the data that are
over the set threshold {threshold}: <br><br>"
160 for i in range(len(all_peaks)):
161     if i != len(all_peaks) - 1:
162         peak_index_1 = all_peaks[i]
163         peak_index_2 = all_peaks[i + 1]
164         messages += f"From {x_axis_name}'s {x_values[soc_peaks[peak_index_1]]} to {
x_values[soc_peaks[peak_index_2]]} ---- Range of {abs(soc_y_values_peaks[peak_index_2] -
soc_y_values_peaks[peak_index_1])}"
165         messages += "<br><br>"
166
167 return x_values[soc_peaks[all_peaks]], soc_y_values_peaks[all_peaks], messages
168
169
170 #ENSURE A STATIC DIRECTORY IS MADE
171 static_dir = os.path.join(os.getcwd(), "static")

```



```

172 if not os.path.exists(static_dir):
173     os.makedirs(static_dir)
174
175
176 #ENSURE THAT THE STATIC DIRECTORY IS EMPTY: When the code is ran, the only file that should
    be
177 #in static is README.pdf. The other files must be residue from the privous time the code was
    ran. As i
178 #discuss more under BACKGROUND FILE REMOVAL right below, this expects the code to be
    continously running
179 files = os.listdir(static_dir)
180 if files:
181     for filename in files:
182         if filename == "README.pdf":
183             continue
184
185         file_path = os.path.join(static_dir, filename)
186         os.remove(file_path)
187
188
189 #--- WEBSITE ---
190
191 app = Flask(__name__)
192 app.secret_key = "secretkey"
193
194
195 #BACKGROUND FILE REMOVAL: Where temp_files_time keeps track of all the temporary files
    created, and when they were created. If the file
196 #is older than 1 hour, it will soon be deleted. It is important to realize that this expects
    the code to be continously running, which
197 #should not be a problem, as servers which host web applications have it continously running
    , so it can reached at any time. It
198 #should also be noted that having files be deleted at a rate too fast could lead to the file
    being unreachable when attempted to be deleted.
199 #This is because it can be in the middle of processing (like generating the graphs...) when
    attempted to be deleted. This should lead to issues
200 #with deletion of all files. Simply quit and re-run the code
201 file_expiry_time = 3600 #1 hour
202 temp_files_time = {}
203 def cleanup_old_files():
204     while True:
205         current_time = time.time()
206         for file_name, creation_time in list(temp_files_time.items()):
207             file_path = os.path.join(os.getcwd(), "static", file_name)
208             if current_time - creation_time > file_expiry_time:
209                 if os.path.exists(file_path):
210                     os.remove(file_path)
211                     del temp_files_time[file_name]
212             time.sleep(5400) #1.5 hours
213
214 cleanup_thread = threading.Thread(target=cleanup_old_files)
215 cleanup_thread.daemon = True
216 cleanup_thread.start()
217
218
219 #THE FIRST PAGE: This is the first page that is presented to the user (route:/). Once a
    delimiter
220 #and number of graphs is submitted, this page reloads to show what has been chosen as the
    delimiter and
221 #number of graphs. When a file is uploaded, THE SECOND PAGE enters
222 #Explaining variables:
223 ###text - A string where it is written all the text that is presented to to the user,
224 #below where the file is chosen and uploaded. That includes a hyperlink to the README
225 ###delimiter_text - A string of the information the user submits in the delimiter text box.
    if
226 #no delimiter is submitted, the delimiter will further be a comma (that is what the
    delimiter
227 #session becomes). If a delimiter is submitted, the submitted will be the delimiter and

```

```

228 delimiter_text
229 #will be updated so the user can understand what currently is the delimiter
230 ###graphs_text - The same as delimiter_text, where the default number of graphs is one
231 @app.route("/")
232 def index():
233     text = """<br><br><br> Upload your .csv formatted file here to be analysed and graphed
234     for you :). <br><br><br>
235     <b>INSTRUCTIONS - READ BEFORE USE:</b> <br><br>
236     <a href="/static/README.pdf" target="_blank">PDF of README</a>"""
237
238     delimiter_text = str(escape(request.args.get("delimiter","")))
239     if delimiter_text:
240         session["delimiter"] = delimiter_text.strip()
241         delimiter_text = "Your chosen delimiter: " + session.get("delimiter")
242     else:
243         session["delimiter"] = ","
244
245     graphs_text = str(escape(request.args.get("num_graphs","")))
246     if graphs_text:
247         session["num_graphs"] = graphs_text
248         graphs_text = "Your chosen number of graphs: " + session.get("num_graphs")
249     else:
250         session["num_graphs"] = "1"
251
252     return (f"""
253         <form action="" method="get">
254
255             Delimiter: <input type="text" name="delimiter">
256
257             <br><br>
258
259             <label for="num_graphs">Number of Graphs:</label>
260             <select id="num_graphs" name="num_graphs"> <!-- First dropdown list -->
261                 {generate_dropdown_options([1,2,3,4,5,6,7,8,9,10])}
262             </select>
263
264             <br><br>
265
266             <input type="submit" value="Submit">
267
268         </form>
269         """
270         + delimiter_text + "<br><br>" + graphs_text + "<br><br><br><br>" +
271         """
272         <form action="/upload" method="post" enctype="multipart/form-data" onsubmit="
273         disableUploadButton()">
274             <input type="file" name="file">
275             <input type="submit" value="Upload" id="uploadButton">
276         </form>
277
278         <script>
279             function disableUploadButton() {
280                 document.getElementById('uploadButton').disabled = true;
281             }
282         </script>
283         """
284         + text)
285
286 #THE SECOND PAGE: This is the second page of the website (route:/upload). When a file is
287 detected,
288 #a temporary file is made, which stores the information of the file. Data is then extracted
289 from the
290 #temporary file, which is used to make dropdown lists of the headers and report if any
291 problems occurred
292 #during the reading of the file. Once the headers on the dropdown lists are picked and
293 submitted , THE
294 #THID PAGE enters. If any problems occur during this process, the except block will

```

```

289 #be entered and the temporary file will be deleted if it exists
290 #Explaining variables:
291 ##subrid / temp_file_path - Absolute paths to the static directory and temporary file,
    respectively
292 ##result - A collection of all the messages made when extracting the data. This can be
    problems that
293 #occurred during the extraction, and information that the file is done being read
294 @app.route("/upload", methods=["POST"])
295 def upload_file():
296     file = request.files.get("file")
297     delimiter= session.get("delimiter", ",")
298     num_graphs = session.get("num_graphs", "1")
299
300     if file is None or file.filename == "":
301         return "No file was selected. Go back and try again"
302
303     if file:
304         try:
305             csv_content = file.stream.read().decode("utf-8-sig")
306
307             subdir = os.path.join(os.getcwd(), "static")
308             temp_file = tempfile.NamedTemporaryFile(delete=False, mode="w", newline="", dir=
subdir)
309
310             #Storing the temporary file along with when it was made in the dictionary
311             temp_files_time[str(temp_file.name)] = time.time()
312             session["temp_file_name"] = temp_file.name
313
314             temp_file.write(csv_content)
315             temp_file.close()
316
317             temp_file_path = os.path.join(subdir, os.path.basename(temp_file.name))
318             with open(temp_file_path) as csv_data:
319                 headers, data, messages = extract_data(csv_data, delimiter)
320
321             result = ""
322             result += "<br><br>".join(messages)
323             result += "<br><br>" + ""
324
325             return (result + "<br><br>" +
326 f"""
327 <form action="/plot" method="post" onsubmit="disableSubmitButton()">
328
329         {generate_dropdowns(int(num_graphs), headers)}
330
331         <input type="submit" value="Submit" id="submitButton">
332         <br><br>
333
334         <b>PLEASE</b> be patient <br><br>
335
336     </form>
337
338     <script>
339         function disableSubmitButton() {{
340             document.getElementById('submitButton').disabled = true;
341         }}
342     </script>
343     """)
344
345     except Exception as e:
346         if "temp_file" in locals():
347             temp_file_path=os.path.join(os.getcwd(), "static",os.path.basename(temp_file
.name))
348
349             if not temp_file.closed:
350                 temp_file.close()
351             if os.path.exists(temp_file_path):
352                 os.remove(temp_file_path)
353             return f"File is unreadable. <br> Make sure to read the instructions on the

```

```

353     front page. Try again <br> ERROR : {e}"
354     else:
355         return f"File is unreadable. <br> Make sure to read the instructions on the front
356         page. Try again"
357
358 #THE THIRD PAGE: This is the third page of the website (route:/plot). Depending on the
359 #number of graphs and
360 #which headers were chosen in THE SECOND PAGE, plots are made accordingly. If the user
361 #wishes to analyze the
362 #state of change of one of the graphs, THE FOURTH PAGE enters. If any problems occur during
363 #this process, the except
364 #block will be entered and the temporary file will be deleted if it is found
365 #Explaining variables:
366 ###plot_html - As stated, how many graphs is made is dependant on the number of graphs. When
367 #each graph is done being made in the
368 #end of the for-loop of this function, they are appended into plot_html in a way that html
369 #understands how to present on a web page. At
370 #the end of each loop, the question to analyze the state of change of that plot is also
371 #appended
372 ###x_axis_name / y_axis_name - The headers that were chosen as the x-axis and y-axis in THE
373 #SECOND PAGE, where the names change depending
374 #on which graph is being plotted in this round of the for-loop
375 ###x_valus / y_values - A list of the values in the column of the chosen header, where if a
376 #value in either columns (x_values OR y_values)
377 #is supposed to be ignored, both lists will not include that data point to ensure correct
378 #dimensions
379 @app.route("/plot", methods=["POST"])
380 def plot_file():
381     try:
382         delimiter= session.get("delimiter", ",")
383         num_graphs = session.get("num_graphs", "1")
384         temp_file_name = session.get("temp_file_name")
385
386         subdir = os.path.join(os.getcwd(), "static")
387         temp_file_path = os.path.join(subdir, os.path.basename(temp_file_name))
388         with open(temp_file_path) as temp_file:
389             headers, data, messages = extract_data(temp_file, delimiter)
390
391         plot_html=""
392         for i in range(int(num_graphs)):
393
394             x_axis_name = str(escape(request.form.get(f"x-axis{i}")))
395             y_axis_name = str(escape(request.form.get(f"y-axis{i}")))
396
397             session[f"x-axis{i}"] = x_axis_name
398             session[f"y-axis{i}"] = y_axis_name
399
400             x_index = headers.index(x_axis_name)
401             y_index = headers.index(y_axis_name)
402             x_values = [p[x_index] for p in data if (p[x_index] != "ignore" and p[y_index]
403             != "ignore")]
404             y_values = [p[y_index] for p in data if (p[y_index] != "ignore" and p[x_index]
405             != "ignore")]
406
407             if any(isinstance(element, datetime) for element in x_values):
408                 binary_conversion(y_values)
409                 fig = px.line(x=x_values, y=y_values)
410             else:
411                 binary_conversion(x_values)
412                 binary_conversion(y_values)
413                 fig = px.line(x=x_values, y=y_values)
414
415             # Customization options
416             fig.update_traces(line=dict(color="#1f77b4"))
417             fig.update_layout(
418                 width=1400, height=700,

```

```

408         plot_bgcolor='whitesmoke',
409         font_color='black',
410         title={
411             'text': f'{y_axis_name} on {x_axis_name}',
412             'font': {
413                 'size': 30,
414                 'color': 'black'
415             }
416         },
417         xaxis=dict(
418             title = x_axis_name,
419             showgrid=True,
420             gridcolor='lightgrey',
421             zerolinecolor = 'black',
422             linecolor='black',
423             linewidth=2,
424             mirror = True,
425             zeroline=False
426         ),
427         yaxis=dict(
428             title = y_axis_name,
429             showgrid=True,
430             gridcolor='lightgrey',
431             zerolinecolor = 'black',
432             linecolor='black',
433             linewidth=2,
434             mirror = True,
435             zeroline=False
436         ),
437     )
438
439     plot_html += pio.to_html(fig, full_html=False) + "<br>"
440     plot_html += f""""Do you wish to analyze the <b>spikes in the data</b>?
441
442         <form action="/soc" method="post" onsubmit = "
443     disableAllSubmitButtons()">
444         Threshold: <input type="text" name="threshold{i}">
445         <input type="submit" value="Submit">
446     </form>
447
448     <br><br><br>""""
449
450     return f""""
451     <b>NOTICE</b> that in the top right corner of each graph you have tools to
452     zoom and adjust the graph to your preference
453
454     <br><br><br>
455
456     <div>{plot_html}</div>
457
458     <script>
459         function disableAllSubmitButtons() {{
460             var buttons = document.querySelectorAll('input[type="submit"]');
461             buttons.forEach(function(button) {{
462                 button.disabled = true;
463             }});
464         }}
465     </script>
466     """"
467
468     except Exception as e:
469         if "temp_file_path" in locals() and os.path.exists(temp_file_path):
470             os.remove(temp_file_path)
471         return f"File could not be read <br>Make sure to read the instructions on the front
472         page. Try again <br> ERROR: {e}"

```

```

#THE FOURTH PAGE: This is the fourth page of the website (route:/soc). A plot of the chosen

```

```

graph and the spikes larger than the user's threshold
473 #found in the data are presented. In addition, text is provided to the user of how many
    spikes there are, and where they are found. Notice that how the graph
474 #is plotted is different here than in THE THIRD PAGE. This is because two plots had to be
    plotted on the same graph. If any problems occur
475 #during this process, the except block will be entered and the temporary file will be
    deleted if it is found
476 #Explaining variables:
477 ###x_spikes / y_spikes - Two lists of peaks which, when plotted together, make the spikes on
    the graph that are larger than the threshold
478 ###message - Contains all the text of how many spikes there are and where in the graph they
    can be found
479 @app.route("/soc", methods=["POST"])
480 def soc():
481     try:
482         delimiter= session.get("delimiter", ",")
483         num_graphs = session.get("num_graphs", "1")
484         temp_file_name = session.get("temp_file_name")
485
486         threshold = 0
487         iteration = 0
488         for i in range(int(num_graphs)):
489             threshold = str(escape(request.form.get(f"threshold{i}")))
490
491             if threshold == "None":
492                 continue
493
494             try:
495                 threshold=float(threshold)
496             except:
497                 return "Not able to turn threshold into a number"
498
499             iteration = i
500             break
501
502             x_axis_name = session.get(f"x-axis{iteration}", "")
503             y_axis_name = session.get(f"y-axis{iteration}", "")
504
505             subdir = os.path.join(os.getcwd(), "static")
506             temp_file_path = os.path.join(subdir, os.path.basename(temp_file_name))
507             with open(temp_file_path) as temp_file:
508                 headers, data, messages = extract_data(temp_file, delimiter)
509
510             os.remove(temp_file_path)
511
512             x_index = headers.index(x_axis_name)
513             y_index = headers.index(y_axis_name)
514             x_values = [p[x_index] for p in data if (p[x_index] != "ignore" and p[y_index] != "
ignore")]
515             y_values = [p[y_index] for p in data if (p[y_index] != "ignore" and p[x_index] != "
ignore")]
516
517             if any(isinstance(element, datetime) for element in x_values):
518                 binary_conversion(y_values)
519                 line_trace = go.Scatter(x=x_values, y=y_values, mode='lines', name=y_axis_name,
line=dict(color="#1f77b4"))
520             else:
521                 binary_conversion(x_values)
522                 binary_conversion(y_values)
523                 line_trace = go.Scatter(x=x_values, y=y_values, mode='lines', name=y_axis_name,
line=dict(color="#1f77b4"))
524
525             # Customization options
526             layout = go.Layout(
527                 width=1400, height=700,
528                 plot_bgcolor='whitesmoke',
529                 font_color="black",
530                 title={

```

```

531         'text': f'{y_axis_name} on {x_axis_name}',
532         'font': {
533             'size': 30,
534             'color': 'black'
535         }
536     },
537     xaxis=dict(
538         title = x_axis_name,
539         showgrid=True,
540         gridcolor='lightgrey',
541         zerolinecolor='black',
542         linecolor='black',
543         linewidth=2,
544         mirror=True,
545         zeroline=False
546     ),
547     yaxis=dict(
548         title = y_axis_name,
549         showgrid=True,
550         gridcolor='lightgrey',
551         zerolinecolor='black',
552         linecolor='black',
553         linewidth=2,
554         mirror=True,
555         zeroline=False
556     ),
557 )
558
559 x_spikes, y_spikes, message = plot_spikes(threshold, x_values, y_values, x_axis_name)
560
561 scatter_trace = go.Scatter(
562     x=x_spikes,
563     y=y_spikes,
564     mode='lines+markers',
565     name='Spikes in Data',
566     line=dict(color='red', dash='dash'),
567     marker=dict(symbol='x-thin-open', color='black', size=8)
568 )
569
570 fig = go.Figure(data=[line_trace, scatter_trace], layout=layout)
571 plot_html = pio.to_html(fig, full_html=False)
572
573 return f"""
574     <b>NOTICE</b> that in the top right corner of each graph you have tools
575     to zoom and adjust the graph to your preference
576
577     <br><br><br>
578     <div>{plot_html}</div>
579
580     <br><br>
581     {message}
582     """
583
584
585 except Exception as e:
586     if "temp_file_path" in locals() and os.path.exists(temp_file_path):
587         os.remove(temp_file_path)
588     return f"File could not be read <br>Make sure to read the instructions on the front
589     page. Try again <br> ERROR: {e}"
590
591
592 if __name__ == "__main__":
593     app.run(host="127.0.0.1", port=80, debug=True)

```

B Lifetime Modeling of Batteries, Oversimplified

```
1 #CODE TO ALGORITHMICLY EVALUATE THE STATE OF HEALTH OF A BATTERY
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import csv
5 import pandas as pd
6 from scipy.signal import find_peaks
7
8 header = []
9 data = []
10
11 filename = '2016-06-03-00-31-19.log'
12
13 #READ THE CSV FILE and append the row of headers into 'header', and the following
14 #rows of data into 'data'
15 with open(filename) as csvfile:
16     csvreader = csv.reader(csvfile, delimiter=";")
17     header = next(csvreader)
18     for datapoint in csvreader:
19         values = [value for value in datapoint]
20         data.append(values)
21
22 #Extract the 1st and 16th column. Since the first column contains dates, it is converted
23 #into
24 #datetime variables using the panda library and pd.to_datetime() to make it possible for
25 #matplotlib to graph the content correctly. The 16th column is the SOC at that exact date
26 date = [(p[0]) for p in data]
27 yaxis = [(p[15]) for p in data]
28 date = pd.to_datetime(date)
29 yaxis = np.array(yaxis, dtype=float)
30
31 #Find all the indexes which are peaks and valleys of the SOC of the battery
32 upper_peaks, _ = find_peaks(yaxis)
33 lower_peaks, _ = find_peaks(-yaxis)
34 all_peaks = sorted(np.append(upper_peaks, lower_peaks))
35 #all_peaks = sorted(np.append(np.append(all_peaks), [len(yaxis)-1]))
36
37 #Find the values at the peaks and valleys
38 all_y_peaks = yaxis[all_peaks]
39
40 #Hysthresis filter the peaks to filter out the small fluctuations which do not contribute to
41 #damage. Explanation
42 #of this algorithm can be found in the main file in 'soc_algorithm' or the README about the
43 #algorithm in the file
44
45 threshold = 5 #ADJUSTABLE, depending on what fluctuation is deemed non-dangerous
46 real_peaks=[]
47 peak_index = 0
48
49 while peak_index < len(all_peaks):
50     if peak_index == 0:
51         next_index = peak_index + 1
52         current_peak_index = peak_index
53         real_peaks.append(all_peaks[current_peak_index])
54         while next_index < len(all_peaks) and abs(all_y_peaks[next_index] - all_y_peaks[
55             current_peak_index]) < threshold:
56             next_index+=1
57             peak_index+=1
58
59         first_round = False
60         peak_index += 1
61         continue
62
63     real_peaks.append(all_peaks[peak_index])
64     #Notice that every peak_index += 1 in this if block signifies skipping over that index,
65     #as we always
```



```

62 #peak_index += 1 at the end of this while loop
63 if peak_index < len(all_y_peaks)-1:
64     if abs(all_y_peaks[peak_index]-all_y_peaks[peak_index+1])<threshold:
65         next_index = peak_index+2
66         current_peak_index = peak_index
67         peak_index += 1
68         if all_y_peaks[current_peak_index+1] > all_y_peaks[current_peak_index]:
69             while(next_index < len(all_peaks) and 0 <= (all_y_peaks[next_index]-
all_y_peaks[current_peak_index]) < threshold):
70                 peak_index+=1
71                 next_index+=1
72             else:
73                 while(next_index < len(all_peaks) and 0 <= (all_y_peaks[current_peak_index]-
all_y_peaks[next_index]) < threshold):
74                     peak_index+=1
75                     next_index+=1
76
77     peak_index += 1
78
79
80 real_peaks = np.array(real_peaks)
81 real_y_peaks = yaxis[real_peaks]
82 real_date_peaks = date[real_peaks]
83
84 #We must refine the filtered peaks by, again, finding its indexes that are peaks
85 #and setting them together in an organized list.
86 upper_peaks, _ = find_peaks(real_y_peaks)
87 lower_peaks, _ = find_peaks(-real_y_peaks)
88 all_peaks = sorted(np.append(upper_peaks, lower_peaks))
89 all_peaks = np.array(sorted(np.append(np.append([0], all_peaks), [len(real_y_peaks)-1])),
dtype = int)
90
91 real_y_peaks = real_y_peaks[all_peaks]
92 real_date_peaks = real_date_peaks[all_peaks]
93
94
95 #THE CORE ALGORITHM. With my given parameters of the amount of spikes to a given DOD
decreases
96 #the SOH from 100 to 80, i was able to both have a logarithmic function which best
approximates
97 #the trend the parameters follow (f(x)) and multiple logarithmic functions witch directly
follow
98 #the given parameters (g(x)) to find how many spikes of any DOD decreases the SOH from 100
to 80.
99 #Given this, by calulating the difference from one SOC peak / valley to the next (the DOD
for
100 #that spike), i was able to find how much the SOC decreases given that one DOD by finding
how many
101 #spikes is needed to decrease the SOH from 100 to 80, and taking the current SOH of the
battery and
102 #and adding -20/(that amount of spikes). This naturally assumes a linear relationship
between how
103 #much health is lost and how many spikes there are of that DOD .
104 SOC = [100,100]
105 days = 2*365
106
107 #Common Exponential
108 def f(x):
109     return -1/0.03*np.log(x/153.21)
110
111 #Different Exponentials
112 def g(x):
113     if 100 >= x >= 80:
114         return -1/0.02*np.log(x/125)
115     elif 80 > x >= 60:
116         return -1/0.02*np.log(x/129.22)
117     elif 60 > x >= 40:
118         return -1/0.03*np.log(x/135)

```

```

119     elif 40 > x >= 20:
120         return -1/0.03*np.log(x/181.49)
121     elif 20 > x >= 10:
122         return -1/0.07*np.log(x/2560)
123     elif 10 > x:
124         return -1/0.02*np.log(x/46.42)
125     else:
126         print("WARNING: DOD larger than 100%. g(x) set as 0")
127         return 0
128
129
130 for j in range(days):
131     for i in range(len(real_y_peaks)-1):
132         diff = abs(real_y_peaks[i+1]-real_y_peaks[i])
133         spikes = f(diff)*1000
134         SOC[0] -= 20/spikes
135
136         spikes = g(diff)*1000
137         SOC[1] -= 20/spikes
138
139 print(f"Exponential with a Common Exponential: {SOC[0]}")
140 print(f"Exponential with Differing Exponential: {SOC[1]}")
141
142 #A SIMPLE LINEAR ALGORITHM
143 SOC = 100
144 hpi = 1/75000
145
146 for j in range(days):
147     for i in range(len(real_y_peaks)-1):
148         diff = abs(real_y_peaks[i+1]-real_y_peaks[i])
149         SOC -= hpi * diff
150
151 print(f"Linear: {SOC}")
152
153 plt.plot(date,yaxis)
154 plt.plot(real_date_peaks,real_y_peaks,"x", color='black')
155 plt.plot(real_date_peaks,real_y_peaks,"--", color='red')
156 plt.xticks(rotation=45)
157 plt.xlabel(str(header[0]))
158 plt.ylabel(str(header[15]))
159 plt.grid(True)
160 plt.show()

```

C Lifetime Modeling of LiFePO4 Batteries

```

1 #CODE TO ALGORITHMICLY EVALUATE THE STATE OF HEALTH OF A BATTERY
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import csv
5 import pandas as pd
6 from scipy.signal import find_peaks
7
8 header = []
9 data = []
10
11 filename = '2016-06-03-00-31-19.log'
12
13 #READ THE CSV FILE and append the row of headers into 'header', and the following
14 #rows of data into 'data'
15 with open(filename) as csvfile:
16     csvreader = csv.reader(csvfile, delimiter=",")
17     header = next(csvreader)
18     for datapoint in csvreader:
19         values = [value for value in datapoint]
20         data.append(values)

```

```

21
22 #Extract the 1st and 16th column. Since the first column contains dates, it is converted
    into
23 #datetime variables using the panda library and pd.to_datetime() to make it possible for
24 #matplotlib to graph the content correctly. The 16th column is the SOC at that exact date
25 date = [(p[0]) for p in data]
26 yaxis = [(p[15]) for p in data]
27 date = pd.to_datetime(date)
28 yaxis = np.array(yaxis, dtype=float)
29
30 #Find all the indexes which are peaks and valleys of the SOC of the battery
31 upper_peaks, _ = find_peaks(yaxis)
32 lower_peaks, _ = find_peaks(-yaxis)
33 all_peaks = sorted(np.append(upper_peaks, lower_peaks))
34 #all_peaks = sorted(np.append(np.append(all_peaks), [len(yaxis)-1]))
35
36 #Find the values at the peaks and valleys
37 all_y_peaks = yaxis[all_peaks]
38
39 #Hysthresis filter the peaks to filter out the small fluctuations which do not contribute to
    damage. Explanation
40 #of this algorithm can be found in the main file in 'soc_algorithm'
41
42 threshold = 5 #ADJUSTABLE, depending on what fluctuation is deemed non-dangerous
43 real_peaks=[]
44 peak_index = 0
45
46 while peak_index < len(all_peaks):
47
48     if peak_index == 0:
49         next_index = peak_index + 1
50         current_peak_index = peak_index
51         real_peaks.append(all_peaks[current_peak_index])
52         while next_index < len(all_peaks) and abs(all_y_peaks[next_index] - all_y_peaks[
current_peak_index]) < threshold:
53             next_index+=1
54             peak_index+=1
55
56         first_round = False
57         peak_index += 1
58         continue
59
60     real_peaks.append(all_peaks[peak_index])
61     #Notice that every peak_index += 1 in this if block signifies skipping over that index,
    as we always
62     #peak_index += 1 at the end of this while loop
63     if peak_index < len(all_y_peaks)-1:
64         if abs(all_y_peaks[peak_index]-all_y_peaks[peak_index+1])<threshold:
65             next_index = peak_index+2
66             current_peak_index = peak_index
67             peak_index += 1
68             if all_y_peaks[current_peak_index+1] > all_y_peaks[current_peak_index]:
69                 while(next_index < len(all_peaks) and 0 <= (all_y_peaks[next_index]-
all_y_peaks[current_peak_index]) < threshold):
70                     peak_index+=1
71                     next_index+=1
72                 else:
73                     while(next_index < len(all_peaks) and 0 <= (all_y_peaks[current_peak_index]-
all_y_peaks[next_index]) < threshold):
74                         peak_index+=1
75                         next_index+=1
76
77     peak_index += 1
78
79
80 real_peaks = np.array(real_peaks)
81 real_y_peaks = yaxis[real_peaks]
82 real_date_peaks = date[real_peaks]

```

```

83
84 #We must refine the filtered peaks by, again, finding its indexes that are peaks
85 #and setting them together in an organized list.
86 upper_peaks, _ = find_peaks(real_y_peaks)
87 lower_peaks, _ = find_peaks(-real_y_peaks)
88 all_peaks = sorted(np.append(upper_peaks, lower_peaks))
89 all_peaks = np.array(sorted(np.append(np.append([0], all_peaks), [len(real_y_peaks)-1])),
    dtype = int)
90
91 real_y_peaks = real_y_peaks[all_peaks]
92 real_date_peaks = real_date_peaks[all_peaks]
93
94 ###
95 #THE LOSS OF STATE OF HEALTH ALGORITHM. FOR EXPLANATION OF THE ALGORITHM, PLEASE
96 #CONSULT THE README
97 ###
98
99 #The different fucntions that apply to Cycles til EOL at 100% DOD depending on the C-rate of
    the ΔSOC. Naturally,
100 #higher C-rates have less cycles until EOL is reached. EOL has been defined as 80% SOH for
    this algorithm
101 def SixC(Temp):
102     return 0.000000569999921*Temp**6 - 0.000067634212089*Temp**5 + 0.002501605233947*Temp**4
        - 0.026162988007769*Temp**3 - 0.078076355454328*Temp**2 + 7.3706551060645*Temp +
        108.10810810810811
103
104 def FiveFiveC(Temp):
105     return 0.000000431133382*Temp**6 - 0.000070328693352*Temp**5 + 0.002762933804923*Temp**4
        - 0.032468092937658*Temp**3 - 0.079545115021854*Temp**2 + 8.389770140326666*Temp +
        111.11111111111111
106
107 def FiveC(Temp):
108     return 0.000000723179799*Temp**6 - 0.000088739762653*Temp**5 + 0.003440359962099*Temp**4
        - 0.038586111955677*Temp**3 - 0.143757912779652*Temp**2 + 9.489091826048348*Temp +
        113.63636363636364
109
110 def FourFiveC(Temp):
111     return 0.000000431133382*Temp**6 - 0.000057697895666*Temp**5 + 0.002395958204267*Temp**4
        - 0.028464257498949*Temp**3 - 0.043675931542431*Temp**2 + 8.333092479483836*Temp +
        116.41
112
113 def FourC(Temp):
114     return 0.000000877074315*Temp**6 - 0.000109420093795*Temp**5 + 0.004158775252525*Temp**4
        - 0.039400703463203*Temp**3 - 0.229843073593074*Temp**2 + 10.0126262626263*Temp +
        121.21212121212122
115
116 def ThreeFiveC(Temp):
117     return 0.000000566436361*Temp**6 - 0.000080990441978*Temp**5 + 0.003392614317104*Temp**4
        - 0.035387669063048*Temp**3 - 0.101458592252962*Temp**2 + 10.141399295431293*Temp +
        129.366
118
119 def ThreeC(Temp):
120     return -0.000000330433455*Temp**6 + 0.000011485042735*Temp**5 + 0.00048344017094*Temp**4
        - 0.015678418803419*Temp**3 + 0.27638888888889*Temp**2 + 8.95299145299145*Temp +
        142.85714285714286
121
122 def TwofiveC(Temp):
123     return 0.00000131008976*Temp**6 - 0.000170466140117*Temp**5 + 0.006427532634244*Temp**4
        - 0.053892141996601*Temp**3 - 0.328702631036326*Temp**2 + 16.44475932176382*Temp +
        164.47368421052633
124
125 def TwoC(Temp):
126     return 0.000041835016835*Temp**5 - 0.005055861646771*Temp**4 + 0.149039638812369*Temp**3
        + 0.29866850321394*Temp**2 - 5.599326599327147*Temp + 224.88521579430855
127
128 def OneFiveC(Temp):
129     return 0.000060315694444*Temp**5 - 0.006026847222222*Temp**4 + 0.131260763888892*Temp**3
        + 1.060451388888866*Temp**2 + 5.611849999999515*Temp + 277.54214285714374

```

```

130
131 def OneC(Temp):
132     if Temp <= 20:
133         return 0.174265*Temp**3 + 0.25735*Temp**2 + 5.147000000000003*Temp + 400
134     else:
135         return -0.033333331666667*Temp**3 + 2.999999833333326*Temp**2 - 119.99999483333309*
Temp + 3466.666616666664
136
137 def ZeroNineC(Temp):
138     if Temp <= 20:
139         return 0.008308275613276*Temp**3 + 2.235419841269841*Temp**2 + 43.28322799422799*
Temp + 446.42857142857144
140     else:
141         return -0.077131029343748*Temp**3 + 8.107980728711645*Temp**2 - 315.1828079551876*
Temp + 5950.239375096354
142
143 def ZeroEightC(Temp):
144     if Temp<=20:
145         return 0.01459037730633*Temp**3 + 2.710626387872001*Temp**2 + 49.70595645516311*Temp
+ 507.61
146     else:
147         return -0.121992193104959*Temp**3 + 13.617668035816976*Temp**2 - 539.1885051616968*
Temp + 9015.343136449519
148
149 def ZeroSevenC(Temp):
150     if Temp<=20:
151         return 0.01650322997416*Temp**3 + 3.574646511627907*Temp**2 + 59.50267700258399*Temp
+ 581.3953488372093
152     else:
153         return -0.19708757020757*Temp**3 + 22.692578754578754*Temp**2 - 901.1881953601954*
Temp + 13856.766300366302
154
155 def ZeroSixC(Temp):
156     if Temp <=20:
157         return 0.015516132996776*Temp**3 + 4.744887830057575*Temp**2 + 72.9825481910056*Temp
+ 684.931506849315
158     else:
159         return -0.300499231950845*Temp**3 + 35.1094470046083*Temp**2 - 1392.857142857143*
Temp + 20384.024577572967
160
161 def ZeroFiveC(Temp):
162     if Temp <=20:
163         return 0.014753746859514*Temp**3 + 6.603255416521479*Temp**2 + 91.1513998215493*Temp
+ 823.0452674897119
164     else:
165         return -0.441848859031522*Temp**3 + 52.19258400372951*Temp**2 - 2075.3627909200663*
Temp + 29570.418494567097
166
167 def ZeroThreeC(Temp):
168     if Temp <= 20:
169         return -0.09865226376989*Temp**3 + 16.07252395348837*Temp**2 + 174.3961659118727*
Temp + 1398.601
170     else:
171         return -1.106628502808827*Temp**3 + 131.5530387149308*Temp**2 - 5194.356026023804*
Temp + 70645.24884644806
172
173 #Cycles til EOL depending on DOD at 1C, 50% middle SOC and 30 C
174 #DOES NOT work for DOD's smaller than 4, just bin it to 4, but have a warning that much
smaller than 4 not accurate
175
176 #Function that gives the amount of Cycles until EOL is reached, depending on the ΔDOD of the
cycles. Note
177 #that this function cannot be applied for ΔDOD smaller than 4%.
178 def CyclesTilEol(DOD):
179     if DOD < 2:
180         print("WARNING! ΔDOD smaller than 2%! Returning value for 2% DOD")
181         return 1881.75 / (1 - 1.01*np.exp(-0.01*2))
182     else:

```

```

183         return 1881.75 / (1 - 1.01*np.exp(-0.01*DOD))
184
185     """if DOD <= 30:
186         if DOD >= 4:
187             return 2577.6 / (1 - 1.06*np.exp(-0.02*DOD))
188         else:
189             print("WARNING! ΔDOD smaller than 4%! Returning value for 4% DOD")
190             return 2577.6 / (1 - 1.06*np.exp(-0.02*4))
191     elif DOD <= 60:
192         return 0.32*DOD**3 - 33.33*DOD**2 + 939.82*DOD + 833.25
193     elif DOD <= 100:
194         return 0.9*DOD**2 - 216.67*DOD + 16972.25
195     else:
196         print("WARNING! ΔDOD higher than 100%. Returning value for 100% DOD")
197         return 0.9*100**2 - 216.67*100 + 16972.25"""
198
199 #Function to bin C-rate into appropriate value, depending on ΔSOC. The function
200 #proceeds to call the relevant function for that C-rate to give Cycles til EOL at
201 #100% DOD, given the temperature during that time of ΔSOC
202 def CyclesTilEolAt100(C_rate,Temp):
203
204     if not -10 <= Temp <= 50:
205         print(f"WARNING! Temperature not within the -10°C - 50°C range.
206             This will lead to an inaccurate estimate of loss of SOH this round, especially
207             if
208             the temprature is far off the range. The temprature was found at: {Temp}*C"")
209
210     if C_rate < 0.4:
211         return ZeroThreeC(Temp)
212     elif C_rate < 0.55:
213         return ZeroFiveC(Temp)
214     elif C_rate < 0.65:
215         return ZeroSixC(Temp)
216     elif C_rate < 0.75:
217         return ZeroSevenC(Temp)
218     elif C_rate < 0.85:
219         return ZeroEightC(Temp)
220     elif C_rate < 0.95:
221         return ZeroNineC(Temp)
222     elif C_rate < 1.25:
223         return OneC(Temp)
224     elif C_rate < 1.75:
225         return OneFiveC(Temp)
226     elif C_rate < 2.25:
227         return TwoC(Temp)
228     elif C_rate < 2.75:
229         return TwofiveC(Temp)
230     elif C_rate < 3.25:
231         return ThreeC(Temp)
232     elif C_rate < 3.75:
233         return ThreeFiveC(Temp)
234     elif C_rate < 4.25:
235         return FourC(Temp)
236     elif C_rate < 4.75:
237         return FourFiveC(Temp)
238     elif C_rate < 5.25:
239         return FiveC(Temp)
240     elif C_rate < 5.75:
241         return FiveFiveC(Temp)
242     elif C_rate < 6.25:
243         return SixC(Temp)
244     else:
245         print("WARNING! C-rate higher than 6C. Returning value for 6C")
246         return SixC(Temp)
247
248 Pack1_Temp = np.array([(p[16]) for p in data], dtype = float)

```

```

249 Pack2_Temp = np.array([(p[18]) for p in data], dtype = float)
250 Pack3_Temp = np.array([(p[20]) for p in data], dtype = float)
251 Pack4_Temp = np.array([(p[22]) for p in data], dtype = float)
252 Pack5_Temp = np.array([(p[24]) for p in data], dtype = float)
253
254
255 daysOfCycleProfile = 365*2
256 healthOfAllPacks = [100, 100, 100, 100, 100]
257 Pack_Temp = [Pack1_Temp, Pack2_Temp, Pack3_Temp, Pack4_Temp, Pack5_Temp]
258
259 #THE MAIN LOOP TO CALCULATE LOSS OF SOH using the previously defined functions
260 for d in range(daysOfCycleProfile):
261     for i in range(len(real_y_peaks)-1):
262         delta_SOC = real_y_peaks[i+1] - real_y_peaks[i]
263         charge = True
264         if delta_SOC < 0:
265             charge = False
266
267         delta_SOC = abs(delta_SOC)
268         delta_t = real_date_peaks[i+1] - real_date_peaks[i]
269         delta_t = delta_t.total_seconds() / 3600
270
271         #Calculated C-rate
272         C_rate = delta_SOC/(100*delta_t)
273
274         #Bottom and Upper indexes of the dates during that ΔSOC. This is used to calculate
275         #the mean temperature during that time frame for the cyclesTilEolAt100() function
276         bottom_index_relevant_dates = real_peaks[all_peaks[i]]
277         upper_index_relevant_dates = real_peaks[all_peaks[i+1]]
278
279
280         for j in range(len(Pack_Temp)):
281             relevant_temperatures = Pack_Temp[j][bottom_index_relevant_dates:
282             upper_index_relevant_dates+1]
283             average_temperature = np.mean(relevant_temperatures)
284
285             cyclesTilEolAt100 = CyclesTilEolAt100(C_rate, average_temperature)
286             howManyMoreCycles = CyclesTilEol(delta_SOC) / CyclesTilEol(100)
287             cyclesTilEol = cyclesTilEolAt100 * howManyMoreCycles
288
289             healthLossOfOneCycle = 20 / cyclesTilEol
290             healthLossOfOneChargeOrDischarge = 0
291             if charge:
292                 healthLossOfOneChargeOrDischarge = (healthLossOfOneCycle*3)/5
293             else:
294                 healthLossOfOneChargeOrDischarge = (healthLossOfOneCycle*2)/5
295
296             healthOfAllPacks[j] -= healthLossOfOneChargeOrDischarge
297
298 print(healthOfAllPacks)
299
300 plt.plot(date,yaxis)
301 plt.plot(real_date_peaks,real_y_peaks,"x", color='black')
302 plt.plot(real_date_peaks,real_y_peaks,"--", color='red')
303 plt.xticks(rotation=45)
304 plt.xlabel(str(header[0]))
305 plt.ylabel(str(header[15]))
306 plt.grid(True)
307 plt.show()

```